PUNTO.TDEA

MANUAL TÉCNICO

CÓMO ESTÁ HECHO

Arquitectura, requisitos de máquina, red, ancho de banda y despliegue. Sirve en cualquier host que cumpla estas capacidades.

TECNOLÓGICO DE ANTIOQUIA

Equipo de ingeniería

Octubre 2026

Contenido

1. Control documental 4

1.1 Control documental . . . 4
1.2 Historial de cambios . . . 4
1.3 Propósito y público . . . 4
1.4 Responsables y periodicidad . . . 4
1.5 Convenciones de nombres . . . 4
1.6 Convenciones de redacción y de comandos . . . 5
1.7 Manejo de información sensible . . . 5
1.8 Herramientas de elaboración . . . 5

2. Objetivo, alcance y descripción 5

2.1 Objetivo del sistema . . . 5
2.2 Alcance de la versión 1 . . . 5
2.3 Descripción general . . . 5

3. Requisitos de hardware, software y red 5

3.1 Ambientes . . . 6
3.2 Requisitos de hardware . . . 6
3.3 Requisitos de software . . . 6
3.4 Requisitos de red . . . 7

4. Arquitectura y componentes 8

4.1 Capas . . . 8
4.2 Diagrama de arquitectura lógica . . . 8
4.3 Componentes . . . 9

5. Instalación, configuración y despliegue 9

5.1 Instalación local de la API y la base . . . 9
5.2 El cliente . . . 10

5.3 Configuración . . . 10
5.4 Integración continua . . . 10
5.5 Despliegue . . . 11
5.6 Plan de rollback . . . 11
5.7 Productos del despliegue . . . 12
5.8 Diagrama de despliegue . . . 12

6. Base de datos y modelo de información 12

6.1 Motor y conexión . . . 12
6.2 Grupos de tablas . . . 13
6.3 Fichas de las tablas que mueven el día . . . 13
6.4 Diccionario breve . . . 13
6.5 Disco y migraciones . . . 14
6.6 Respaldo de la base . . . 14

7. Integraciones y APIs 14

7.1 API de casos . . . 14
7.2 Tiempo real . . . 15
7.3 Servicios externos . . . 15

8. Seguridad, perfiles y permisos 15

8.1 Perfiles . . . 15
8.2 Autenticación . . . 15
8.3 Autorización . . . 15
8.4 Datos personales . . . 15
8.5 Secretos . . . 15

9. Operación, monitoreo y recuperación 15

9.1 Qué está cubierto hoy . . . 16
9.2 Disponibilidad . . . 16

9.3 Respaldo y restauración 16
9.4 Ante una falla 16
10. Incidentes, mantenimiento y glosario 16
10.1 Runbooks cortos 16
10.2 Mantenimiento 17
10.3 Pendientes conocidos 17
10.4 Glosario 17

1. Control documental

1.1 Control documental

Campo	Valor
Código	PTDA-MT-001
Título	Manual técnico de Punto TdeA
Versión	1.0
Fecha	Octubre 2026
Estado	Vigente
Sistema	Punto TdeA, atención por WhatsApp

1.2 Historial de cambios

Versión	Fecha	Qué cambió
1.0	Octubre 2026	Primera edición: arquitectura, requisitos, datos y operación.

1.3 Propósito y público

Este manual es para el equipo de ingeniería que instala, cambia o recupera Punto TdeA. Describe cómo está hecho el sistema que ya corre, qué máquina y qué red necesita, y qué no se debe romper. El manual de usuario, PTDA-MU-001, explica las mismas pantallas sin este detalle.

1.4 Responsables y periodicidad

Quien publica un cambio de arquitectura, de variables o de modelo de datos actualiza este manual en el mismo cambio. No hay un calendario fijo: se toca cuando el código o el despliegue dejan de coincidir con lo escrito aquí.

1.5 Convenciones de nombres

- El ticket visible es TDEA- y seis dígitos, tomado del id de la conversación.
- Las rutas de producto van en español: /bandeja/, /campanas/, /avisos/, /base-de-conocimiento/.
- La API de casos cuelga de /api/cases/. El webhook de WhatsApp es /api/whatsapp/webhook/.
- Hay dos modelos Contact. No se unifican: uno es la persona del caso y el otro es la audiencia de campañas.

1.6 Convenciones de redacción y de comandos

Los comandos van en bloque verde, listos para copiar en un servidor Linux. Una ruta entre paréntesis angulares es un valor que pone el equipo, no un literal. Si este texto y el código discrepan, manda el código.

1.7 Manejo de información sensible

Este manual no trae contraseñas, tokens ni documentos de estudiantes. Los secretos viven en el entorno del servidor, nunca en el repositorio. No se registran en logs el token de Meta, la clave de Django, cadenas de conexión ni el teléfono completo si se puede evitar.

1.8 Herramientas de elaboración

Se armó a partir del código, de los estilos de la interfaz y de capturas del sistema en marcha. La marca es la de la aplicación: fondo #0a0a0a, verde #3DFF8A, Anton y Montserrat. No hay un archivo de logo suelto; el símbolo se dibuja como en la interfaz.

2. Objetivo, alcance y descripción

2.1 Objetivo del sistema

Punto TdeA concentra la atención del Tecnológico de Antioquia que llega por WhatsApp: el asistente responde con la base de conocimiento, y si hace falta una persona el caso entra a la bandeja. Además permite campañas, avisos y un dashboard de la cola.

2.2 Alcance de la versión 1

- Incluye bandeja, dashboard, conocimiento, campañas, avisos, notificaciones y páginas de error.
- El canal de casos en operación es WhatsApp. El modelo admite otros canales, pero la interfaz está centrada en ese.
- Un archivo adjunto sin texto se guarda en la bandeja y no se envía a Meta. El texto sí sale por WhatsApp.
- No incluye una aplicación móvil propia. Quien atiende usa el navegador. Quien escribe usa WhatsApp.

2.3 Descripción general

Es un solo proyecto Django. Las apps de producto son cases, knowledge, communications, announcements, dashboard y notifications. El paquete PuntoTdeA guarda configuración, login, permisos de módulo y las páginas de error. El proceso público es Daphne, porque la bandeja y las notificaciones usan WebSocket.

3. Requisitos de hardware, software y red

Los tamaños de esta sección son una propuesta para el piloto. No son un contrato con un proveedor ni un ancho de banda garantizado. Se ajustan cuando haya tráfico real. Sirven igual en una máquina propia o en un servicio que entregue las mismas capacidades.

3.1 Ambientes

Ambiente	Para qué	Datos
Desarrollo	Cambiar y probar en la máquina del equipo.	SQLite si no hay base, o el compose local.
Integración	Probar el mismo contenedor que saldrá a operación.	Copia de la base, sin tokens de producción.
Operación	Atender a la comunidad.	PostgreSQL y Redis propios. Secretos reales.

3.2 Requisitos de hardware

3.2.1 Estación de desarrollo

Un computador con navegador actual, Python 3.12 y, si se usa el compose, Docker. No hace falta un teléfono para desarrollar la bandeja: el webhook se puede probar con la firma de Meta en las pruebas automáticas.

3.2.2 Dispositivos de quien escribe

La persona de la comunidad no instala Punto TdeA. Escribe por WhatsApp en su teléfono. Ese tráfico va a Meta. A nuestros servidores solo llega el webhook, que es un JSON pequeño, y de vuelta sale el texto de la respuesta.

3.2.3 Servidores de operación

Dos roles. En el piloto pueden vivir en la misma máquina. En operación conviene separarlos para respaldar la base sin tocar la aplicación. El disco de datos va aparte del disco del sistema, para ampliarlo y restaurarlo sin reinstalar el sistema operativo.

Rol	vCPU	RAM	Disco
Aplicación	2	4 GB	30 GB para el sistema y la imagen.
Datos	2	4 GB	30 GB de sistema y 32 GB para PostgreSQL.
Redis	Comparte la aplicación	1 GB alcanza en el piloto	No guarda la operación.

> Redis no es la base de datos. Si se pierde, la bandeja deja de empujar mensajes en vivo hasta que vuelve; los casos siguen en PostgreSQL.

3.3 Requisitos de software

Software	Versión	Para qué
Linux x64	Ubuntu Server 24.04 LTS u otro equivalente	Sistema de los dos roles.
Docker Engine y Compose v2	Estable actual	Correr la imagen de la aplicación.
Imagen de Punto TdeA	La del release	Python 3.12, Django 5.1 y Daphne. No se instala el runtime fuera de la imagen.
PostgreSQL	15	Base de la operación. El compose local usa postgres:15.
Redis	7	Canal de WebSocket. El compose local usa redis:7.
Python en CI	3.12	Las pruebas de GitHub Actions. No usar 3.14 para ese trabajo.

El navegador del asesor es el único cliente. No hay tienda de aplicaciones ni un paquete móvil que publicar.

3.4 Requisitos de red

Solo la entrada web es pública. La base y Redis escuchan en la red privada. El puerto 5432 no se abre a internet.

Origen	Destino	Puerto	Para qué
Internet	Aplicación	443 HTTPS	Asesores, API y webhook. Por dentro el proceso usa 8000 si hay un proxy.
Aplicación	PostgreSQL	5432 TCP	Consultas. Solo desde la red de la aplicación.
Aplicación	Redis	6379 TCP	Tiempo real. Solo desde la red de la aplicación.
Equipo admin	Servidores	22 SSH	Administración, solo desde IPs autorizadas.
Aplicación	Internet	443 HTTPS	graph.facebook.com: envíos y lectura de plantillas.

Dominio y WebSocket

Hace falta un nombre público con certificado TLS en la entrada, no dentro del contenedor. La bandeja y las notificaciones usan WebSocket. El proxy tiene que dejar pasar esa mejora de conexión y mantenerla viva más tiempo que el ping del cliente. Si el proxy corta a los 30 segundos, la bandeja parece congelada aunque el proceso siga sano.

Ancho de banda

Punto TdeA mueve poco por sus propios servidores. El chat de la comunidad viaja por WhatsApp. Aquí entran páginas del asesor, JSON del webhook y, de vez en cuando, un adjunto de hasta 25 MB que se queda en el disco de la aplicación.

Tramo	Mínimo propuesto	Por qué
Asesor y aplicación	10 Mbps	Páginas, WebSocket y un adjunto ocasional.
Aplicación y PostgreSQL	100 Mbps, misma red	Muchas consultas pequeñas.
Aplicación y Redis	100 Mbps, misma red	Avisos de mensajes nuevos.
Aplicación y Meta	10 Mbps sostenidos	Webhook y textos. Un archivo grande es un pico, no el promedio.
Teléfono y WhatsApp	4G o wifi de la persona	Ese tramo no pasa por nuestros servidores.

> Estas cifras son una estimación del equipo para el piloto. Se validan con el tráfico de entrada y de salida del servidor cuando haya uso real, no con un número de catálogo. Conviene que el audio y las imágenes de WhatsApp sigan saliendo de Meta y no se reenvíen desde el disco de la aplicación.

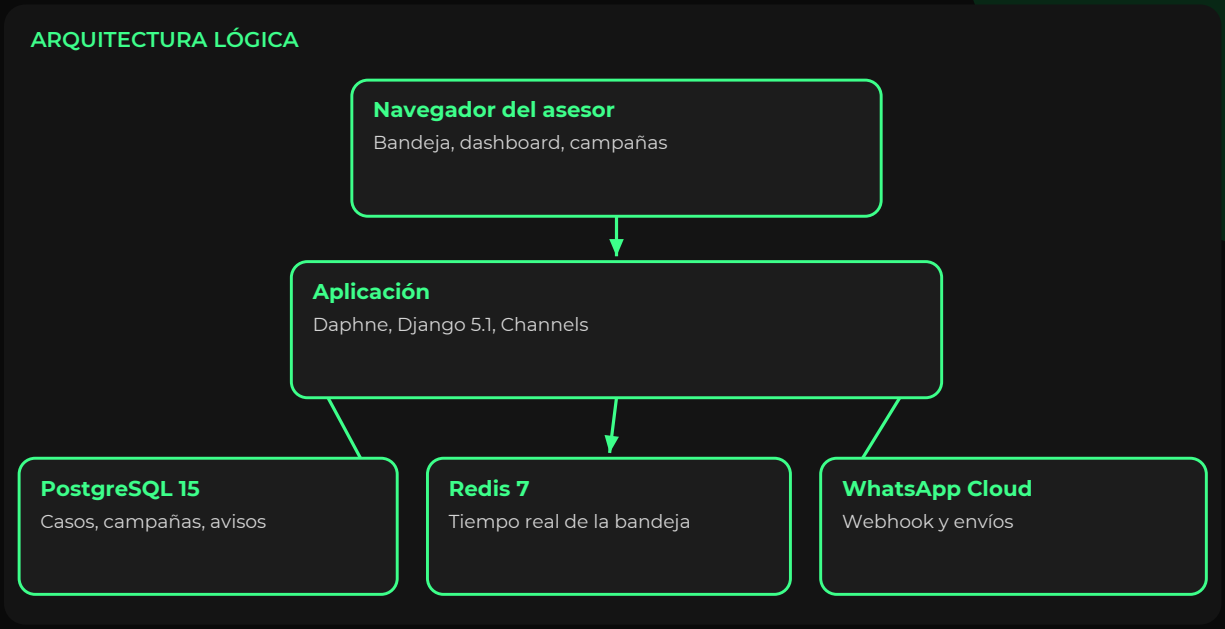
4. Arquitectura y componentes

4.1 Capas

Capa	Qué hay
Presentación	Plantillas Django, CSS propio y el navegador del asesor. Landing en /.
Aplicación	Django 5.1, DRF y Channels. Daphne sirve HTTP y WebSocket.
Datos	PostgreSQL 15 en operación. SQLite solo si no hay DB_HOST ni DATABASE_URL.
Tiempo real	Redis si hay REDIS_URL. Si no, capa en memoria: no sirve con más de un proceso.
Integraciones	Un cliente hacia graph.facebook.com: communications/adapters/meta_adapter.py.
Seguridad	Sesión de Django, permisos por módulo y secretos en el entorno.
Operación	Migraciones al arrancar, health del webhook y un proceso aparte para campañas programadas.

4.2 Diagrama de arquitectura lógica

El asesor solo habla con la aplicación. La aplicación habla con la base, con Redis y con Meta. No hay un segundo webhook: el alias de conocimiento llama la misma función y no se configura en Meta.



4.3 Componentes

Componente	Responsabilidad
cases	Bandeja, tickets, mensajes, adjuntos y cierre.
knowledge	Artículos, FAQ y el asistente que decide si abre un caso.
communications	Audiencias, plantillas de Meta, campañas y el cliente de WhatsApp.
announcements	Avisos y, si está configurado, su publicación en estados.
notifications	Campana y lista de avisos para el equipo.
dashboard	Números de la cola. No escribe casos.
PuntoTdeA	Settings, login, permisos, errores y el arranque ASGI.

```
WhatsApp
  -> POST /api/whatsapp/webhook/
  -> communications.webhooks.meta_webhook
       estados -> BroadcastRecipient
       texto -> knowledge.chatbot.whatsapp.handle_message
                    si escala -> cases.ingest_inbound_message
  -> /bandeja/ claim -> send_text_message
  -> telefono del cases.Contact
```

5. Instalación, configuración y despliegue

5.1 Instalación local de la API y la base

El compose levanta PostgreSQL 15, Redis 7, Daphne en el puerto 8000 y el proceso que revisa campañas cada 30 segundos. Las credenciales de ese compose son de desarrollo. No se reutilizan en operación.

```
docker compose up -d --build
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
```

Sin Docker, con el entorno del repositorio y sin DB_HOST, Django usa SQLite. Alcanza para pruebas de pantalla. No representa la operación.

```
python -m venv venv
./venv/bin/pip install -r requirements.txt
./venv/bin/python manage.py migrate
./venv/bin/daphne -b 127.0.0.1 -p 8000 PuntoTdeA.asgi:application
```

5.2 El cliente

No hay una aplicación que instalar en el teléfono del equipo. El asesor abre el navegador, entra en la raíz del sitio y trabaja en el menú. Quien escribe desde la comunidad ya tiene WhatsApp.

5.3 Configuración

La aplicación no debe arrancar en operación sin estas variables. Lo marcado como secreto no se commitea ni se pega en este manual.

Variable	Secreto	Para qué
DJANGO_DEBUG	No	false en operación.
DJANGO_SECRET_KEY	Sí	Obligatoria si DEBUG es false.
DJANGO_ALLOWED_HOSTS	No	Dominio público, separado por comas.
DATABASE_URL o DB_*	Sí la clave	PostgreSQL. Si faltan, cae a SQLite.
REDIS_URL	No	Canal de WebSocket.
META_ACCESS_TOKEN	Sí	Enviar y leer plantillas.
WHATSAPP_PHONE_NUMBER_ID	No	Número de Cloud API.
WHATSAPP_BUSINESS_ACCOUNT_ID	No	Crear plantillas. Sin esto, Enviar a Meta no puede.
META_APP_SECRET	Sí	Firma del webhook.
META_VERIFY_TOKEN	Sí	Verificación GET del webhook.
INTERNAL_API_TOKEN	Sí	La UI de campañas lee segmentos con este token.
MEDIA_UPLOADS_ENABLED	No	false corta las subidas de adjuntos.

> Python no lee un archivo .env por su cuenta. Lo sustituye el compose o el entorno del host. Después de cambiar una variable hay que recrear el proceso. Un reinicio que no recarga el entorno deja el valor viejo.

5.4 Integración continua

GitHub Actions corre en cada push y pull request a dev y a main, sobre Ubuntu, con Python 3.12. Hace check --deploy, collectstatic y manage.py test. No llama a Meta: el envío se parcha y el webhook se prueba con la firma. Un despliegue no debería salir si ese check está en rojo.

5.5 Despliegue

Estado y ventana

El contenedor es la unidad que se publica. El proveedor concreto no cambia los pasos: hace falta un host con la capacidad de la sección 3, un volumen para la base y otro si se quieren conservar adjuntos. La ventana la acuerda el equipo. Migrar con gente en la bandeja es el momento delicado, no el reinicio de Daphne en sí.

Orden de arranque

01 Red
Abrir solo los puertos de la sección 3.4. El 5432 y el 6379 quedan privados.

02 Datos
Instalar PostgreSQL 15, montar el disco de datos y crear una base y un usuario de la aplicación. No usar el superusuario postgres desde Django.

03 Redis
Levantarlo en la red de la aplicación.

04 Secretos
Cargarlos en el entorno del host. Ninguno va en la imagen.

05 Imagen
Construir la imagen del release. El Dockerfile ya deja los estáticos dentro.

06 Migraciones
El arranque las aplica y aborta si no puede. No se sirve tráfico con un esquema viejo.

07 Procesos
Daphne en el puerto interno, y un segundo proceso: python manage.py run_scheduled_campaigns --interval 30. Sin ese segundo proceso, lo programado no sale.

08 Meta
Apuntar el webhook a /api/whatsapp/webhook/ con el token de verificación. Una sola URL.

Validación

Prueba	Resultado esperado
Login de un asesor	Entra al dashboard y el menú muestra solo sus módulos.
GET /api/whatsapp/health/	Responde sano.
Bandeja	Carga y el WebSocket no se cae al minuto.
Puerto 5432 desde internet	Sin respuesta.
Cierre de un caso de prueba	El caso queda cerrado aunque Meta rechace el aviso.

5.6 Plan de rollback

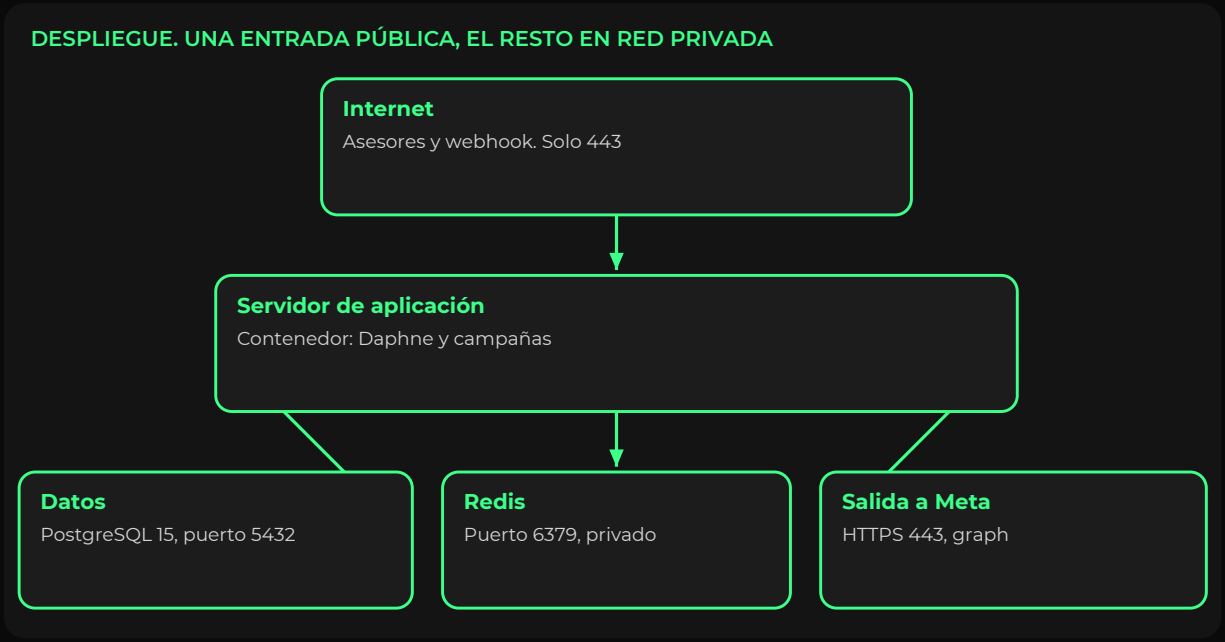
- Antes de migrar, pg_dump de la base. Sin ese archivo no hay vuelta atrás segura.
- Conservar la imagen anterior. Si el health falla, volver a esa imagen.
- Volver el código sin restaurar la base solo es válido si las migraciones nuevas no rompen el código viejo. Si lo rompen, se restaura el dump.
- Los adjuntos están en disco, en media/. Un contenedor nuevo sin ese volumen los pierde aunque la base vuelva.

5.7 Productos del despliegue

- Imagen del release identificada por el commit.
- Base migrada y usuario de aplicación, no el superusuario.
- Variables cargadas en el host.
- Webhook de Meta apuntando a una sola URL.
- Volumen de PostgreSQL y, si hay adjuntos, volumen de media/.

5.8 Diagrama de despliegue

El tráfico de asesores y el webhook entran solo por HTTPS. La base y Redis no tienen dirección pública. La salida a Meta es la única conexión de la aplicación hacia internet además de la respuesta a quien ya entró.



La demo local es el mismo dibujo en una sola máquina: compose publica el 8000 en el computador, Postgres no se publica y Redis sí queda en 6379 para desarrollo. Esa publicación de Redis no se copia a operación.

6. Base de datos y modelo de información

6.1 Motor y conexión

PostgreSQL 15, puerto 5432, usuario propio de la aplicación. DATABASE_URL tiene prioridad sobre DB_HOST, DB_NAME, DB_USER, DB_PASSWORD y DB_PORT. Sin ninguna de las dos, el proceso usa SQLite y no es operación. La conexión se reutiliza hasta 10 minutos.

6.2 Grupos de tablas

Grupo	Tablas	Para qué
Cuenta	auth_user y sesiones de Django	Asesores. No hay un modelo User propio.
Caso	Channel, Department, Contact, Conversation, Message, CaseComment, MessageAttachment, ReplyTemplate	La bandeja y el ticket.
Conocimiento	Category, Intent, KnowledgeArticle, FAQ, ChatConversation	Lo que responde el asistente y el estado del flujo.
Campaña	Contact, AudienceSegment, MessageTemplate, Campaign, BroadcastRecipient y el resto de communications	Audiencia y envío masivo. Su Contact no es el del caso.
Aviso	Announcement, StatusAd	Comunicado e intento de publicarlo en estados.
Aviso interno	Notification, NotificationRead	La campana del equipo.

6.3 Fichas de las tablas que mueven el día

cases.Conversation

Un caso. Canal, contacto, external_thread_id, ticket_number, estado (pendiente, completado, rechazado, escalado, cerrado), prioridad, dependencia, tema, assigned_to, claimed_at, claimed_by, closed_at, closed_by y last_message_at. En WhatsApp el hilo externo es el número.

cases.Message y MessageAttachment

Message guarda dirección, cuerpo, external_id y sent_at. El adjunto apunta al mensaje, guarda el archivo en media/cases/attachments/ y el tipo: imagen, video u ofimática. Tope 25 MB. Las imágenes de avisos y de FAQ no van aquí: viven en la propia fila, en la base.

cases.Contact y communications.Contact

Los dos tienen nombre, documento, correo, teléfono, programa y semestre. El de cases es la persona del ticket, con documento único. El de communications es la audiencia: suma rol, whatsapp_opt_in, attributes y source. El opt-in de campañas no convierte a esa fila en el estudiante del caso. El asistente busca por teléfono en communications y copia datos al crear el ticket.

communications.Campaign y BroadcastRecipient

Campaign guarda plantilla, segmento, estado, horario y cupo por minuto. BroadcastRecipient es una persona de ese envío: teléfono congelado, estado, id del proveedor y error. El id del proveedor lo escribe el envío, no una carga manual.

knowledge.ChatConversation

Estado del asistente por persona: flow_state, última pregunta, si ya hay caso y por qué escaló. Cuando escala, case_conversation apunta al ticket. A partir de ahí el texto nuevo se anexa al caso y el bot no sigue contestando.

6.4 Diccionario breve

Tabla	Clave de negocio
Channel	code único: whatsapp, facebook, instagram, web.
Department	Dependencia a la que se escala.
CaseComment	Nota interna. No sale por WhatsApp.
ReplyTemplate	Texto rápido de la bandeja. No es plantilla de Meta.
MessageTemplate	Plantilla de Meta: cuerpo, encabezado y botones.
AudienceSegment	Reglas o archivo de una audiencia.
WhatsAppWebhookEvent	Evento ya visto. Evita procesar dos veces el mismo id.
Announcement	Título, texto, imagen e urgente.
Notification	Módulo, evento, título y URL para la campana.

El detalle de cada columna está en los modelos. Si esta ficha y el modelo no coinciden, manda el modelo.

6.5 Disco y migraciones

- media/ guarda adjuntos de casos. Hay que montarlo como volumen.
- Las imágenes de avisos y de FAQ están en la base y viajan con el dump.
- Las migraciones las aplica el arranque, con reintentos si la base todavía no acepta conexiones. A la décima falla, el contenedor no queda sirviendo.
- No hay un paquete de datos semilla de producción. Un superusuario inicial solo se crea si el entorno trae DJANGO_SUPERUSER_USERNAME, EMAIL y PASSWORD, y no pisa la clave de uno que ya existe.

6.6 Respaldo de la base

pg_dump de la base de la aplicación, todos los días, guardado fuera del mismo disco de datos. Probar un restore en otro PostgreSQL 15 antes de necesitarlo. Redis no se respalda. media/ se copia junto con el dump si hay adjuntos que no se pueden perder.

7. Integraciones y APIs

7.1 API de casos

Cuelga de /api/cases/ y exige sesión con permiso del módulo. La documentación viva está en /api/docs/.

Método y ruta	Uso
GET /api/cases/tickets/	Lista de la bandeja.
POST /api/cases/tickets/{id}/claim/	Tomar.
POST /api/cases/tickets/{id}/reply/	Responder, con attachments.
POST /api/cases/tickets/{id}/close/	Cerrar.
GET /api/cases/tickets/{id}/messages/	Hilo.
GET y POST .../comments/	Notas internas.
GET /api/cases/attachments/{id}/download/	Bajar un adjunto. Exige el permiso del módulo.

La ingesta es idempotente por external_message_id. Tomar un caso ya asignado responde 409. Cerrar uno ya cerrado responde 200 y no manda otro aviso. Si Meta rechaza el saludo o el aviso de cierre, el

caso no se deshace.

7.2 Tiempo real

WebSocket en ws/bandeja/ y en la conversación abierta. Notificaciones en ws/notifications/. Si se quita el permiso a mitad de la suscripción, el consumidor de cases cierra con 4403. Sin Redis, dos procesos no se ven entre sí.

7.3 Servicios externos

El único servicio externo de producto es la API de WhatsApp de Meta, en graph.facebook.com, versión v21.0 salvo que el entorno diga otra. Un solo cliente envía texto y plantillas. Los encabezados con archivo exigen header_media_url; sin URL, el adaptador no envía. Nadie más del proyecto llama a graph.facebook.com.

8. Seguridad, perfiles y permisos

8.1 Perfiles

Quién	Qué puede
Asesor con permiso del módulo	Ver y trabajar ese módulo. El menú oculta el resto.
Staff	Puede seguir un caso que ya tiene dueño y borrar un caso.
Superusuario	Pasa los permisos de módulo. Sigue siendo una cuenta de Django.
Sin sesión	La página va al login. La API responde 401.

8.2 Autenticación

Login en /login/ con usuario o correo. Recordarme fija 14 días; si no, la cookie muere al cerrar el navegador. El parámetro next solo se honra si es una ruta relativa que empieza por una barra y no por dos.

8.3 Autorización

module_access_required exige el permiso app.access_app, directo o por grupo. Sin permiso, la página es el 403 de error, no un texto plano. La API responde JSON. Los nombres son cases.access_cases, knowledge.access_knowledge, communications.access_communications y announcements.access_announcements. Tener view_ en el admin no abre el módulo.

8.4 Datos personales

El sistema guarda nombre, documento, correo, teléfono, programa y semestre. Eso identifica a una persona. El acceso es por cuenta y por permiso de módulo. No se escriben en los logs. Este manual no sustituye el registro ni la política de tratamiento de la institución: solo fija lo que el software ya hace.

8.5 Secretos

Viven en el entorno del host. El repositorio puede nombrar la variable, nunca el valor. Rotar META_ACCESS_TOKEN o DJANGO_SECRET_KEY obliga a recargar el proceso. La clave de firma de sesión, si cambia, cierra todas las sesiones.

9. Operación, monitoreo y recuperación

9.1 Qué está cubierto hoy

- El arranque migra y levanta Daphne. Si la base no responde, reintenta y aborta.
- Hay un health del webhook en /api/whatsapp/health/.
- Las pruebas de lectura de bandeja y de avisos existen en la rama pruebas. No sustituyen manage.py test y no pegan a Meta, ni toman, ni responden, ni cierran.
- No hay un tablero de métricas de infraestructura dentro de la aplicación. El dashboard es de la cola de casos, no del servidor.

9.2 Disponibilidad

Un solo proceso Daphne atiende HTTP y WebSocket. Si se cae, cae la bandeja y el webhook. El objetivo de operación es tener el health en verde y el WebSocket estable. No hay una cifra de disponibilidad comprometida en el código: eso lo fija quien opera el host.

9.3 Respaldo y restauración

```
pg_dump -h 10.0.2.4 -U puntotdea_app -d puntotdea -Fc -f puntotdea.dump
pg_restore -h 10.0.2.4 -U puntotdea_app -d puntotdea --clean --if-exists puntotdea.dump
```

La dirección del ejemplo es privada. En restauración, parar la aplicación antes, restaurar, y solo entonces volver a levantarla. Revisar que el esquema coincida con la imagen que va a arrancar.

9.4 Ante una falla

01	**Mirar el health** Si no responde, el proceso no está sirviendo. Revisar el log de Daphne, no el dashboard.
02	**Mirar la base** Si el health vive pero la bandeja da 500, probar la conexión a PostgreSQL desde el contenedor.
03	**Mirar Redis** Si la bandeja carga y no se actualiza sola, Redis no está o hay más de un proceso sin él.
04	**Volver a la imagen anterior** Solo después del dump, y con la regla de rollback de la sección 5.6.

10. Incidentes, mantenimiento y glosario

10.1 Runbooks cortos

Síntoma	Dónde mirar
La bandeja no recibe WhatsApp	URL única del webhook, META_VERIFY_TOKEN y WHATSAPP_PHONE_NUMBER_ID.
El aviso amarillo de plantillas	Faltan META_ACCESS_TOKEN o WHATSAPP_BUSINESS_ACCOUNT_ID.
Una campaña programada no sale	El segundo proceso run_scheduled_campaigns no está corriendo.
403 en un módulo	Falta el permiso access_ del módulo. Cerrar sesión y volver a entrar después de asignarlo.
404 con página verde	Ruta mal escrita. La API y el admin no usan esa página.
Adjuntos desaparecen al redesplegar	media/ no está en un volumen.
Cerrar el caso no avisa al teléfono	Meta rechazó el texto. El caso sí quedó cerrado. Revisar el error guardado, no reabrir por eso.

10.2 Mantenimiento

- Preventivo: dump diario, espacio del disco de datos, y que el certificado TLS no esté por vencer.
- Correctivo: imagen anterior más dump, en ese orden. No se edita la base a mano para arreglar un caso.
- Al cambiar código de negocio, tocarlo en cases/services, no en el router. Un wamid nuevo por cada salida de la misma conversación.

10.3 Pendientes conocidos

- Un adjunto sin texto no sale por WhatsApp. Queda en la bandeja.
- El documento de arquitectura todavía nombra a Pascual Bravo en un párrafo. La marca de la interfaz y de este manual es Tecnológico de Antioquia.
- Las pruebas de carga no están en la rama de integración. Viven en la rama pruebas.

10.4 Glosario

Término	Significado
Bandeja	Cola de chats de WhatsApp.
Caso o ticket	Una Conversation. El número visible es TDEA- y seis dígitos.
Tomar	claim. Asigna el caso y manda el saludo.
Plantilla rápida	ReplyTemplate. Texto de la bandeja, local.
Plantilla de Meta	MessageTemplate. La tiene que aprobar Meta antes de una campaña.
Audiencia	Segmento de communications.Contact con opt-in.
Aviso	Announcement. No es una notificación de la campana.
Webhook	POST de Meta a /api/whatsapp/webhook/. Uno solo.